

15. ROS2 Time-related API

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. Time-related API Overview

The time-related APIs in ROS2 include Rate, Time, Duration, and operations between Time and Duration. Let's explain them one by one.

- First, create a package to store related program files:

```
ros2 pkg create learning_time --build-type ament_python --dependencies rclpy
```

2. create_rate

ROS2 also provides the create_rate function, which is a tool for **controlling loop execution frequency**. Its core function is to allow a piece of code to **execute periodically at a fixed frequency**. `Rate` ensures loop execution frequency stability by controlling the loop's "sleep time". **Important:** `Rate` generally cannot be used directly in the main thread, as it will permanently block callback events. It is usually only used in programs with multi-threaded callbacks or in sub-threads.

Working principle:

1. Record the start time of each loop iteration;
2. Execute the code within the loop;
3. Calculate the difference between the actual time consumed by the current loop and the "target interval time";
4. Automatically sleep for the corresponding difference time, ensuring that the interval from the start of one loop to the start of the next loop strictly equals the "target interval" (e.g., 100 milliseconds).

Although both `Rate` and `Timer` can achieve periodic execution, they are suitable for different scenarios:

Feature	<code>Rate</code>	<code>Timer</code>
Implementation	Based on active sleep within the loop (blocks current thread)	Based on callback function (non-blocking, triggered by ROS 2 event loop)
Applicable Scenarios	Suitable for loops that need to execute at fixed frequency in the same thread (e.g., main control logic)	Suitable for periodic tasks that need asynchronous execution (does not block main thread)
Flexibility	Can directly control flow within the loop (e.g., break to exit)	Needs to control callback execution through flags or other means

- Below is the basic usage of `Rate`, implementing a loop that executes twice per second:
 - Create a file `rate_demo.py` in the package:

```
import rclpy
from rclpy.node import Node
import threading

class RateExampleNode(Node):
    def __init__(self):
        super().__init__("rate_example_node")
        self.get_logger().info("Rate example node started")

    def run_loop(self):
        # Use node's create_rate() to create a 2Hz Rate
        rate = self.create_rate(2)

        count = 0
        try:
            while rclpy.ok():
                self.get_logger().info(f"Loop executed {count} times")
                count += 1
                rate.sleep() # Sleep until next cycle (0.5 seconds)
        except KeyboardInterrupt:
            self.get_logger().info("Loop interrupted")

def main(args=None):
    rclpy.init(args=args)
    node = RateExampleNode()

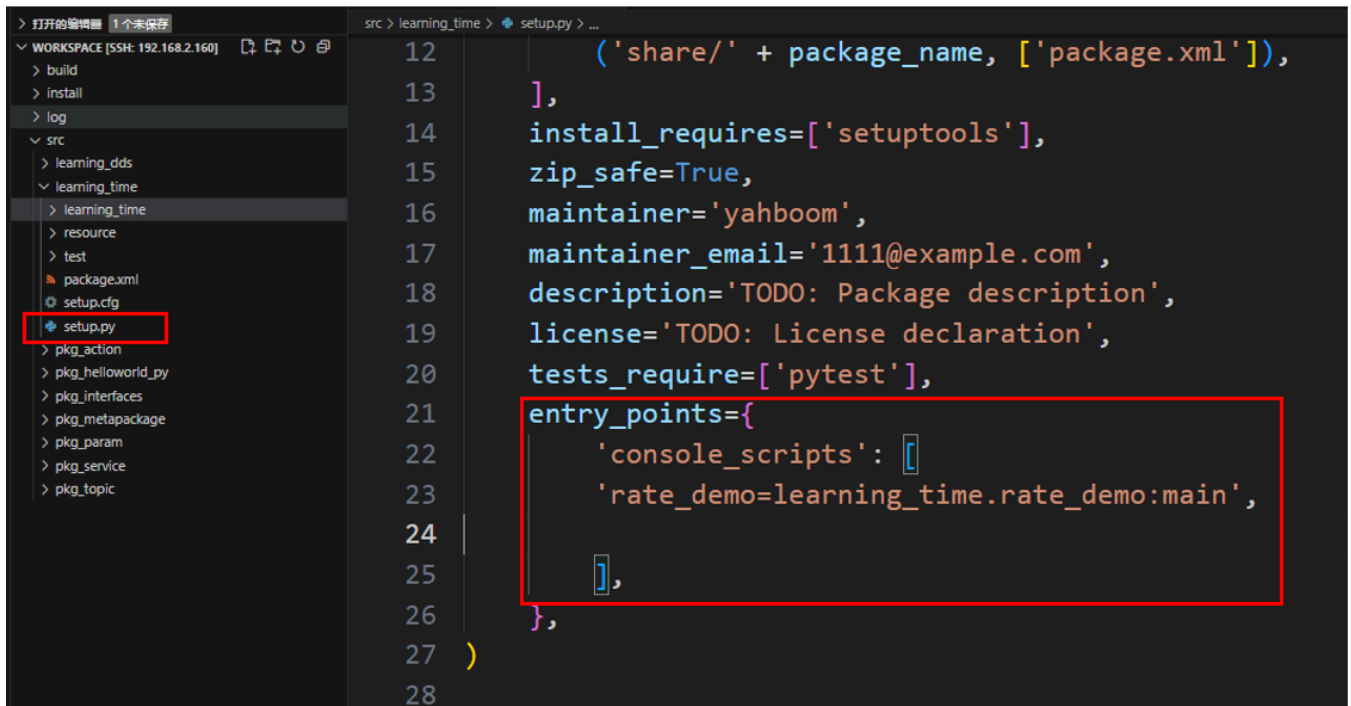
    # Create thread to run loop (avoid blocking main thread)
    loop_thread = threading.Thread(target=node.run_loop)
    loop_thread.start()

    # Main thread executes spin, maintaining ROS 2 node operation
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        loop_thread.join() # wait for thread to finish
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()
```

- Configure the corresponding `setup.py` file. Add to `console_scripts`:

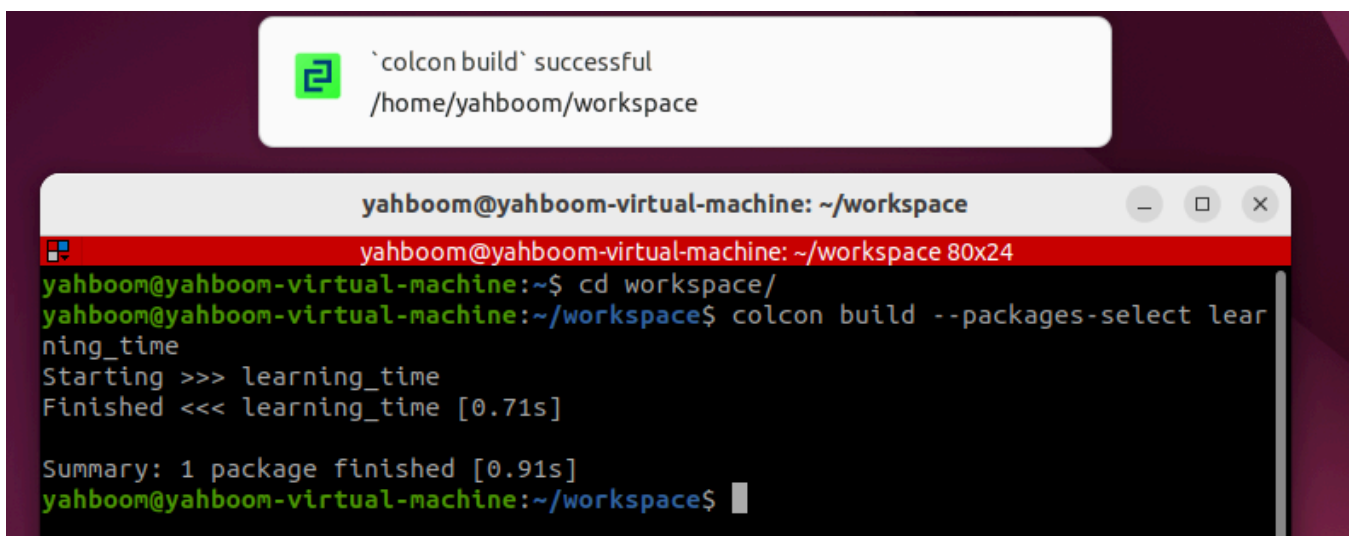
```
'rate_demo=learning_time.rate_demo:main'
```



```
12 ('share/' + package_name, ['package.xml']),
13 ],
14 install_requires=['setuptools'],
15 zip_safe=True,
16 maintainer='yahboom',
17 maintainer_email='1111@example.com',
18 description='TODO: Package description',
19 license='TODO: License declaration',
20 tests_require=['pytest'],
21 entry_points={
22     'console_scripts': [
23         'rate_demo=learning_time.rate_demo:main',
24     ],
25 },
26 },
27 )
28
```

- Build the package:

```
colcon build --packages-select learning_time
```



```
yahboom@yahboom-virtual-machine: ~/workspace
`colcon build` successful
/home/yahboom/workspace

yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine:~$ cd workspace/
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select learning_time
Starting >>> learning_time
Finished <<< learning_time [0.71s]

Summary: 1 package finished [0.91s]
yahboom@yahboom-virtual-machine:~/workspace$
```

- Refresh workspace environment and run node:

```
source ./install/setup.bash
```

```
ros2 run learning_time rate_demo
```

```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 80x24
yahboom@yahboom-virtual-machine:~/workspace$ source install/setup.bash
yahboom@yahboom-virtual-machine:~/workspace$ ros2 run learning_time rate_demo
[INFO] [1757059176.053501355] [rate_example_node]: Rate 示例节点启动
[INFO] [1757059176.054427139] [rate_example_node]: 循环执行 0 次
[INFO] [1757059176.555647565] [rate_example_node]: 循环执行 1 次
[INFO] [1757059177.056046927] [rate_example_node]: 循环执行 2 次
[INFO] [1757059177.555834344] [rate_example_node]: 循环执行 3 次
[INFO] [1757059178.055316796] [rate_example_node]: 循环执行 4 次
[INFO] [1757059178.555822847] [rate_example_node]: 循环执行 5 次
[INFO] [1757059179.055854307] [rate_example_node]: 循环执行 6 次
[INFO] [1757059179.556198596] [rate_example_node]: 循环执行 7 次
[INFO] [1757059180.055669535] [rate_example_node]: 循环执行 8 次
[INFO] [1757059180.555035501] [rate_example_node]: 循环执行 9 次
[INFO] [1757059181.054900909] [rate_example_node]: 循环执行 10 次
[INFO] [1757059181.556449990] [rate_example_node]: 循环执行 11 次
[INFO] [1757059182.056311527] [rate_example_node]: 循环执行 12 次
[INFO] [1757059182.555986180] [rate_example_node]: 循环执行 13 次
[INFO] [1757059183.055526793] [rate_example_node]: 循环执行 14 次
[INFO] [1757059183.555156111] [rate_example_node]: 循环执行 15 次
```

3. Timer Application

- Timer is used to create a timer that triggers periodically to execute some periodic tasks
- Example: Create two timers. One executes once per second to print the execution count, and another executes once every 0.5 seconds to print the current time.
- Create a new program file Timer_demo.py:

```
import rclpy
from rclpy.node import Node

class TimerDemoNode(Node):
    def __init__(self):
        super().__init__('timer_demo_node')

        # Counter for demonstrating timer execution count
        self.counter = 0

        # Create timer: Execute callback function once per second
        self.timer = self.create_timer(1.0, self.timer_callback)

        # Create a faster timer: Execute once every 0.5 seconds
        self.fast_timer = self.create_timer(0.5, self.fast_timer_callback)

        self.get_logger().info("Timer node started")

    def timer_callback(self):
        """1-second timer callback function"""
        self.counter += 1
        current_time = self.get_clock().now()
```

```

        # Print current time and counter value
        self.get_logger().info(
            f"[1-second timer] Execution {self.counter}, current time:
{current_time.seconds_nanoseconds()}"
        )

    def fast_timer_callback(self):
        """0.5-second timer callback function"""
        # Print current timestamp (nanoseconds)
        self.get_logger().info(
            f"[0.5-second timer] Current timestamp: {self.get_clock().now().nanoseconds}"
        )

def main(args=None):
    # Initialize ROS 2
    rclpy.init(args=args)

    # Create node
    node = TimerDemoNode()

    # Run node
    rclpy.spin(node)

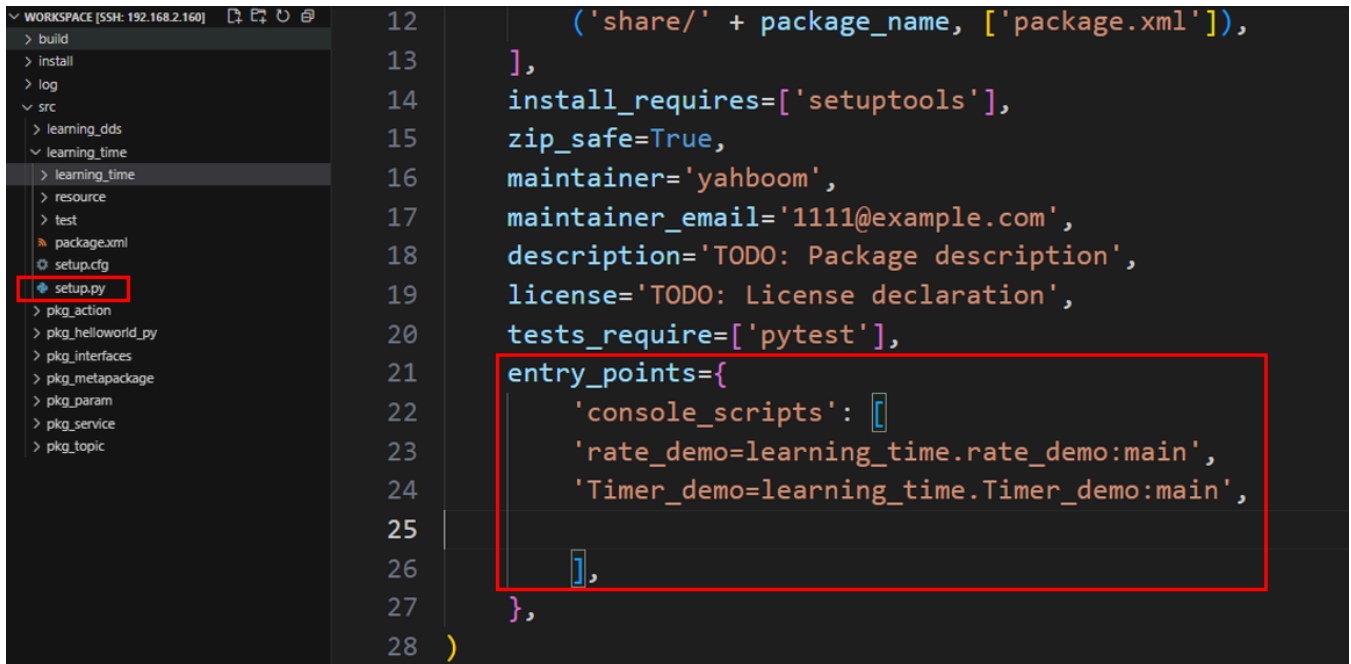
    # Shutdown ROS 2
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

- Configure the corresponding setup.py file. Add to console_scripts:

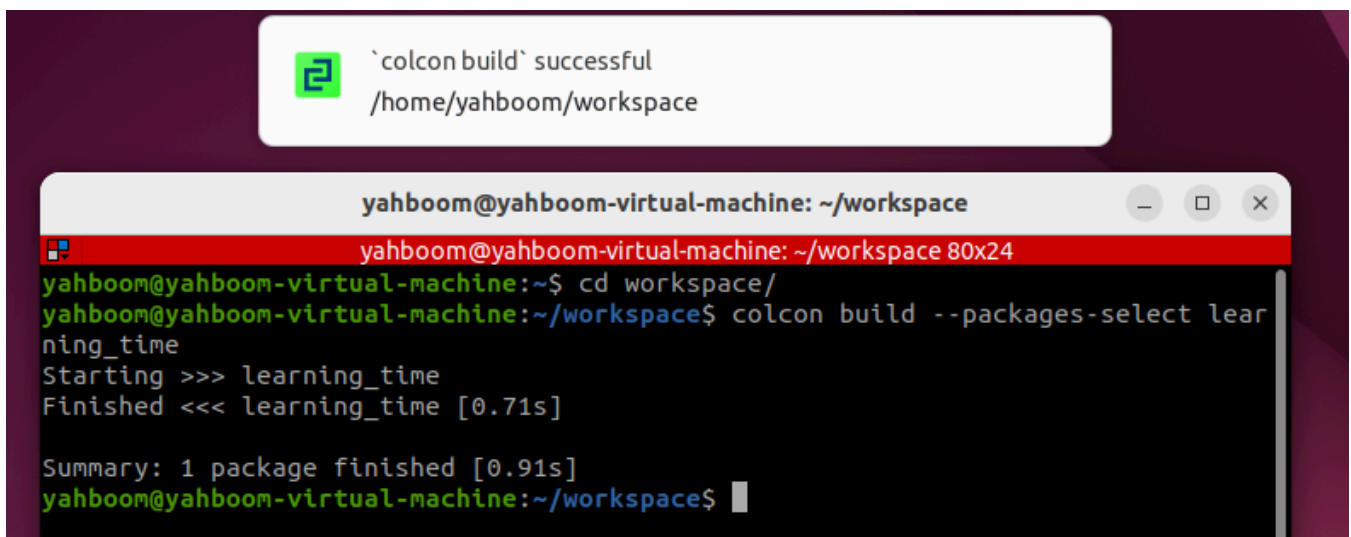
```
'Timer_demo=learning_time.Timer_demo:main'
```



```
12     ('share/' + package_name, ['package.xml']),
13 ],
14 install_requires=['setuptools'],
15 zip_safe=True,
16 maintainer='yahboom',
17 maintainer_email='1111@example.com',
18 description='TODO: Package description',
19 license='TODO: License declaration',
20 tests_require=['pytest'],
21 entry_points={
22     'console_scripts': [
23         'rate_demo=learning_time.rate_demo:main',
24         'Timer_demo=learning_time.Timer_demo:main',
25     ],
26 },
27 },
28 )
```

- Build the package:

```
colcon build --packages-select learning_time
```



```
colcon build` successful
/home/yahboom/workspace

yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 80x24
yahboom@yahboom-virtual-machine:~$ cd workspace/
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select learning_time
Starting >>> learning_time
Finished <<< learning_time [0.71s]

Summary: 1 package finished [0.91s]
yahboom@yahboom-virtual-machine:~/workspace$
```

- Refresh workspace environment and run node:

```
source ./install/setup.bash
```

```
ros2 run learning_time Timer_demo
```

```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 80x24
yahboom@yahboom-virtual-machine:~/workspace$ source install/setup.bash
yahboom@yahboom-virtual-machine:~/workspace$ ros2 run learning time Timer_demo
[INFO] [1757059329.395564545] [timer_demo_node]: 定时器节点已启动
[INFO] [1757059329.891631857] [timer_demo_node]: [0.5秒定时器] 当前时间戳: 1757059329891008756
[INFO] [1757059330.391806054] [timer_demo_node]: [1秒定时器] 第 1 次执行, 当前时间: (1757059330, 391143464)
[INFO] [1757059330.392652648] [timer_demo_node]: [0.5秒定时器] 当前时间戳: 1757059330392110483
[INFO] [1757059330.891208466] [timer_demo_node]: [0.5秒定时器] 当前时间戳: 1757059330890472233
[INFO] [1757059331.391747680] [timer_demo_node]: [1秒定时器] 第 2 次执行, 当前时间: (1757059331, 391145049)
[INFO] [1757059331.392596890] [timer_demo_node]: [0.5秒定时器] 当前时间戳: 1757059331392112596
[INFO] [1757059331.891140285] [timer_demo_node]: [0.5秒定时器] 当前时间戳: 1757059331890548604
[INFO] [1757059332.391227028] [timer_demo_node]: [1秒定时器] 第 3 次执行, 当前时间: (1757059332, 391146640)
```

- As you can see, the timer triggers according to the set timing. Each callback function prints corresponding log information.

4. get_clock - Get Current Time

- The get_clock function can be used to get the clock object, and then the current time can be obtained through the () method.
- Create a new program file get_clock_demo.py and fill in the following example program:

```
import rclpy
from rclpy.node import Node
from rclpy.time import Time

class TimeExampleNode(Node):
    def __init__(self):
        super().__init__("time_example_node")

        # Get the node's clock object (uses system clock by default)
        self.clock = self.get_clock()

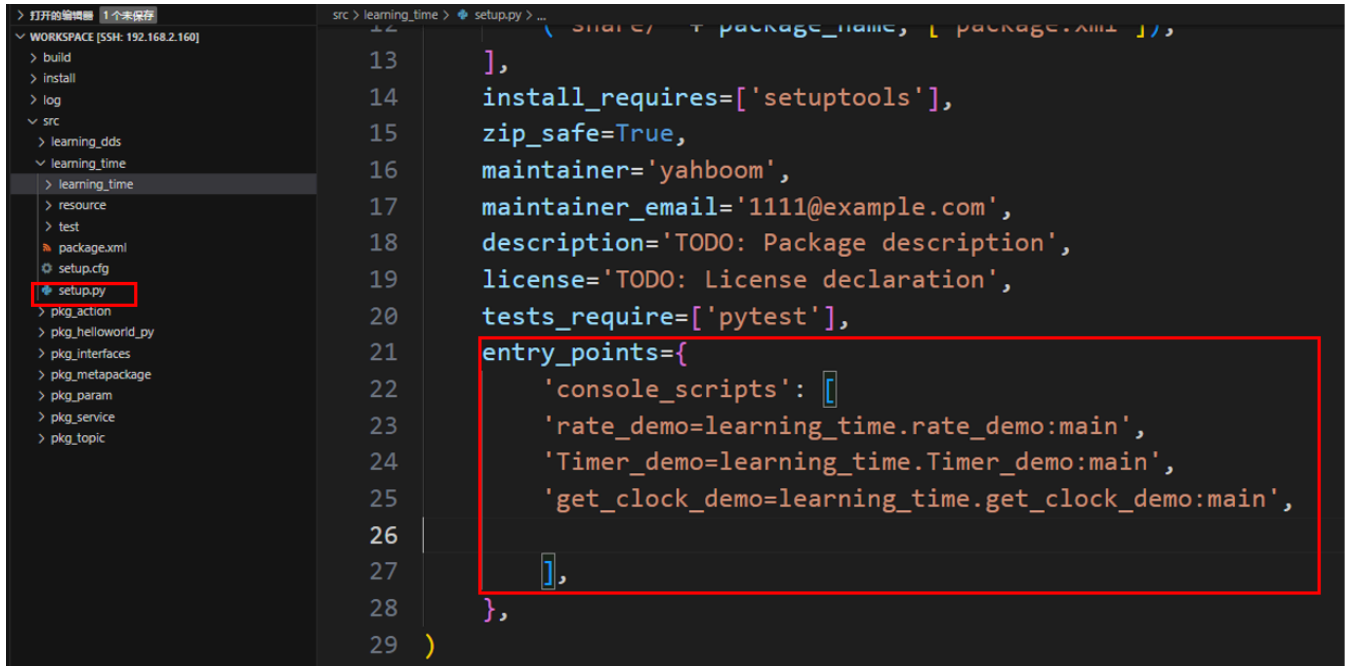
        # Get current time (returns Time object)
        current_time = self.clock.now()
        self.get_logger().info(f"Current time: {current_time}")

def main(args=None):
    rclpy.init(args=args)
    node = TimeExampleNode()
    rclpy.spin_once(node) # Run node once
    node.destroy_node()
    rclpy.shutdown()
```

```
if __name__ == "__main__":
    main()
```

- Configure the corresponding setup.py file. Add to console_scripts:

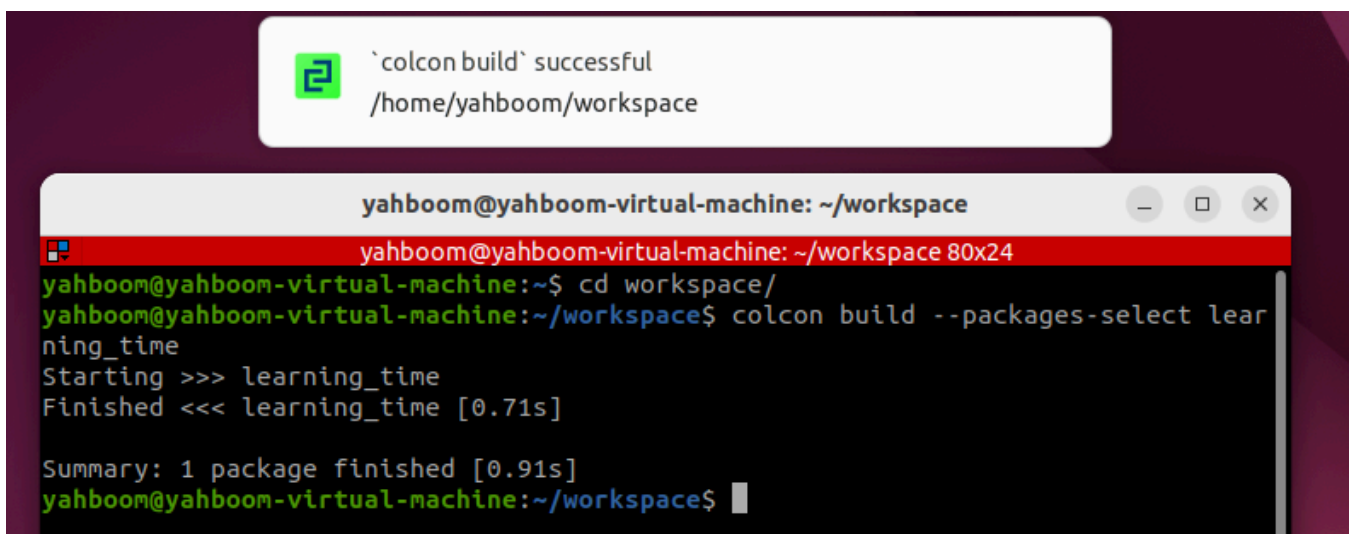
```
'get_clock_demo=learning_time.get_clock_demo:main'
```



```
13 ],
14 install_requires=['setuptools'],
15 zip_safe=True,
16 maintainer='yahboom',
17 maintainer_email='1111@example.com',
18 description='TODO: Package description',
19 license='TODO: License declaration',
20 tests_require=['pytest'],
21 entry_points={
22     'console_scripts': [
23         'rate_demo=learning_time.rate_demo:main',
24         'Timer_demo=learning_time.Timer_demo:main',
25         'get_clock_demo=learning_time.get_clock_demo:main',
26     ],
27 },
28
29 )
```

- Build the package:

```
colcon build --packages-select learning_time
```



```

`colcon build` successful
/home/yahboom/workspace

yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 80x24
yahboom@yahboom-virtual-machine:~$ cd workspace/
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select learning_time
Starting >>> learning_time
Finished <<< learning_time [0.71s]

Summary: 1 package finished [0.91s]
yahboom@yahboom-virtual-machine:~/workspace$
```

- Refresh workspace environment and run node:

```
source ./install/setup.bash
```

```
ros2 run learning_time get_clock_demo
```

```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 93x24
yahboom@yahboom-virtual-machine:~/workspace$ ros2 run learning_time get_clock_demo
[INFO] [1757059580.529675470] [time_example_node]: 当前时间: Time(nanoseconds=1757059580522331199, clock_type=ROS_TIME)
```

5. Time and Duration

- The `Time` class in ROS is used to represent a specific **point in time** (e.g., "2023-10-01 12:00:00"). It is usually used to mark the **moment** when an event occurs.
- The `Duration` class represents the **interval** between two points in time (e.g., "5 seconds"). It is used to calculate time differences or delays.
- **Example:** Time and Duration application
- Create a new program file `TimeDuration_demo.py`:

```
import rclpy
from rclpy.time import Time
from rclpy.duration import Duration

def main():
    rclpy.init()
    node = rclpy.create_node("time_opt_node")

    # How to use Time class: Create 'point in time, moment'
    time1 = Time(seconds=10)
    time2 = Time(seconds=4)
    # How to use Duration class: Create 'duration, time period'
    duration1 = Duration(seconds=3)
    duration2 = Duration(seconds=5)

    # Time points can be compared
    node.get_logger().info("time1 >= time2 ? %d" % (time1 >= time2))
    node.get_logger().info("time1 < time2 ? %d" % (time1 < time2))

    # Time periods and time points can do math operations
    t3 = time1 + duration1
    t4 = time1 - time2
    t5 = time1 - duration1

    node.get_logger().info("t3 = %d" % t3.nanoseconds)
    node.get_logger().info("t4 = %d" % t4.nanoseconds)
    node.get_logger().info("t5 = %d" % t5.nanoseconds)

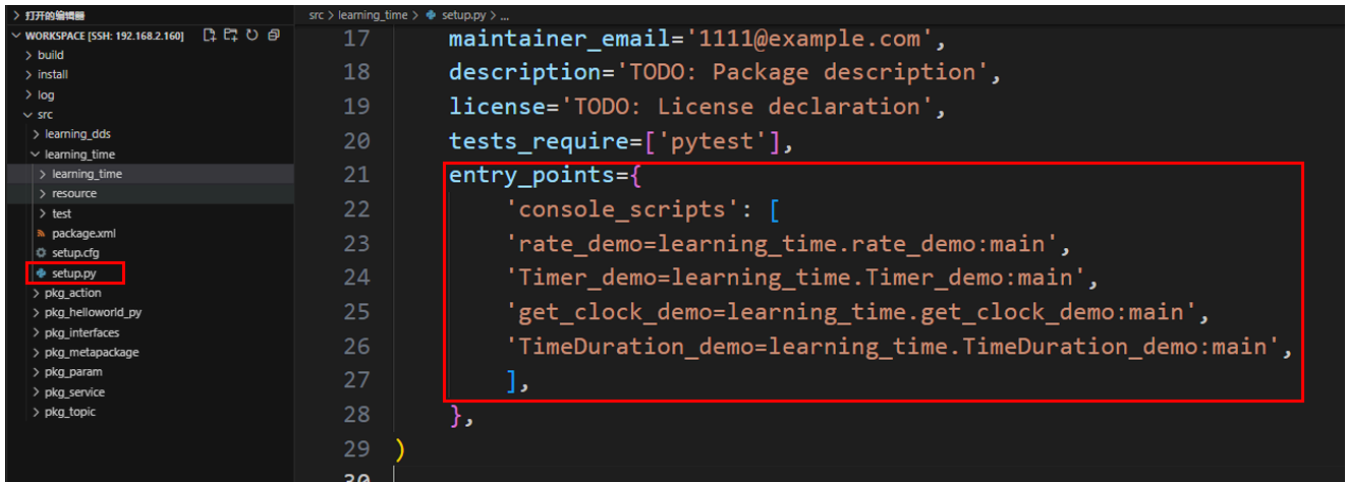
    # Time periods can be compared
    node.get_logger().info("-" * 80)
    node.get_logger().info("duration1 >= duration2 ? %d" % (duration1 >= duration2))
    node.get_logger().info("duration1 < duration2 ? %d" % (duration1 < duration2))

    rclpy.shutdown()
```

```
if __name__ == "__main__":
    main()
```

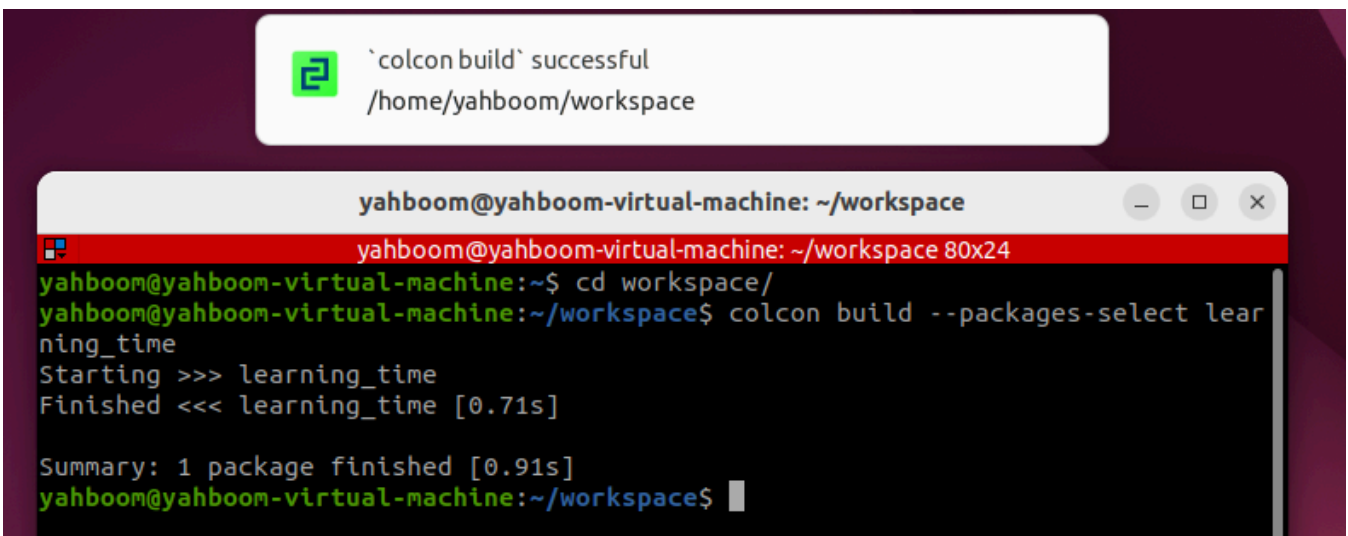
- Configure the corresponding setup.py file. Add to console_scripts:

```
'TimeDuration_demo=learning_time.TimeDuration_demo:main'
```



- Build the package:

```
colcon build --packages-select learning_time
```



- Refresh workspace environment and run node:

```
source ./install/setup.bash
```

```
ros2 run learning_time TimeDuration_demo
```

```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 93x24
yahboom@yahboom-virtual-machine:~/workspace$ source install/setup.bash
yahboom@yahboom-virtual-machine:~/workspace$ ros2 run learning_time TimeDuration_demo
[INFO] [1757059710.444443545] [time_opt_node]: time1 >= time2 ? 1
[INFO] [1757059710.444712497] [time_opt_node]: time1 < time2 ? 0
[INFO] [1757059710.444938448] [time_opt_node]: t3 = 13000000000
[INFO] [1757059710.445120564] [time_opt_node]: t4 = 60000000000
[INFO] [1757059710.445298614] [time_opt_node]: t5 = 70000000000
[INFO] [1757059710.445483789] [time_opt_node]: -----
[INFO] [1757059710.445674891] [time_opt_node]: duration1 >= duration2 ? 0
[INFO] [1757059710.445859100] [time_opt_node]: duration1 < duration2 ? 1
yahboom@yahboom-virtual-machine:~/workspace$
```

- This example demonstrates that time points and time periods can perform mathematical operations, allowing us to flexibly query and manipulate data with different timestamps.